Infosys | Information Security Group

Assessment Name: **AgentOS**

Assessment ID	22311631
Scan Type	Application Vulnerability Assessment
VA Scope	Enterprise
Assessment Initiated Date	10/13/2026 03:07:42 PM
Assessment Initiated By	Kadam, Siddharth
Assessor	Kadam, Siddharth
Total No.Of Vulnerabilities	16
Auditee	Kanade, Trupti
Location	Hyderabad
Assessment Closure Date	10/21/2026 12:00:00 PM
Tools Used	BurpSuite
Type of Assessment	Black Box(DAST)

COPYRIGHT NOTICE

This documentation is the property of Infosys Limited. All ideas and information contained within these documents are the intellectual property rights of Infosys Limited. These documents are not for general distribution and are meant for use only for the person they are specifically issued to. Copying or unauthorized distribution of these documents, in any form or means including electronic, mechanical, photocopying or otherwise is illegal.

Infosys Limited
Hosur Road
Electronic City,
Bangalore 560 100 India.
Website: http://www.infosys.com

Table of Contents

SUMMARY..... 4

OBJECTIVE..... 4

ABOUT INFORMATION SECURITY GROUP..... 4

SCOPE..... 4

RISK CLASSIFICATION..... 4

OUT OF SCOPE..... 5

SUMMARY OF VULNERABILITIES FOUND BY SEVERITY..... 5

CONCLUSION..... 14

REFERENCES..... 15

Summary

This exercise has been carried out to understand the security posture of the **"AgentOS"** application from a Vulnerability assessment perspective.

Objective

To conduct a Vulnerability assessment on the **"AgentOS"** application and to provide the stakeholders a security posture of the application. Security assessment is conducted on the application through the "**BurpSuite**" tool.

About Information Security Group

Information Security Group (ISG) is a group dedicated to information security related activities at Infosys. Performing Vulnerability Assessments and Penetration Testing on applications and infrastructure is one of the activities undertaken by us.

Scope

The scope of the engagement is as described below.

Application Vulnerability Assessment - Review the security of the in-scope application by performing a Automated and Manual vulnerability exercise.

Risk Classification

Below table describes the aspects taken into consideration before classifying the risk of vulnerability.

Type	Description
Critical	Absence or weakness in controls will cause a severe impact to the organization and its assets/resources under protection
High	Absence or weakness in controls will cause a significant impact to the organization and its assets/resources under protection
Medium	Absence or weakness in controls will cause a major impact to the organization and its assets/resources under protection
Low	Absence or weakness in controls will cause a minor impact to the organization and its assets/resources under protection

SCOPE OF ASSESSMENT

I. WEB APP AUTOMATED TESTS:

- OWASP Top 10-2025
  1. A01:2025 – Broken Access Control
  2. A02:2025 – Security Misconfiguration
  3. A03:2025 – Software Supply Chain Failures
  4. A04:2025 – Cryptographic Failures
  5. A05:2025 – Injection
  6. A06:2025 – Insecure Design
  7. A07:2025 – Authentication Failures
  8. A08:2025 – Software or Data Integrity Failures
  9. A09:2025 – Security Logging and Alerting Failures
  10. A10:2025 – Mishandling of Exceptional Conditions

Out of Scope

- Vulnerability Assessment of Infrastructure
- Any vulnerability fixes and regression test
- Any physical access layer test

Summary of Vulnerabilities Found by Severity

Vulnerabilities Reported Below :

Vulnerability ID	Vulnerability-464292
Vulnerability Title	Cross-origin resource sharing
Vulnerability Description from Scan Results	An HTML5 cross-origin resource sharing (CORS) policy controls whether and how content running on other domains can perform two-way interaction with the domain that publishes the policy. The policy is fine-grained and can apply access controls per-request based on the URL and other features of the request. If another domain is allowed by the policy, then that domain can potentially attack users of the application. If a user is logged in to the application, and visits a domain allowed by the policy, then any malicious content running on that domain can potentially retrieve content from the application, and sometimes carry out actions within the security context of the logged in user. Even if an allowed domain is not overtly malicious in itself, security vulnerabilities within that domain could potentially be leveraged by an attacker to exploit the trust relationship and attack the application that allows access. CORS policies on pages containing sensitive information

	should be reviewed to determine whether it is appropriate for the application to trust both the intentions and security posture of any domains granted access.
Auditor Recommendation from Scan Results	Any inappropriate domains should be removed from the CORS policy.
Severity	Critical
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Changed the default to empty (deny) in `shared/config.py` and `server.py` so no CORS header is emitted unless an origin is explicitly configured, restricted allowed methods and headers to verified values, and documented the variable in `config.env.example`.

Vulnerability ID	Vulnerability-464303
Vulnerability Title	Sensitive Information Exposure in JWT Payload
Vulnerability Description from Scan Results	The JWT payload contains user-related information such as email address, role, tenant details, and authorization attributes. JWT contents are Base64 encoded and can be decoded by any user possessing the token. Exposing sensitive information within the token payload may result in information disclosure and provide valuable data to attackers regarding user roles, application structure, and authorization mechanisms.
Auditor Recommendation from Scan Results	• Minimize the information stored within JWT payloads. • Avoid storing sensitive user or business information unless strictly required. • Store only essential claims necessary for authentication and authorization. • Consider replacing sensitive attributes with reference identifiers. • Enforce token expiration, rotation, and secure transmission using HTTPS. • Periodically review token contents to ensure compliance with data minimization principles.
Severity	High
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Reduced the token payload to non-sensitive claims only (`sub`, `sid`, `jti`, `type`, `iat`, `exp`) in `auth/tokens.py`, with the remaining attributes resolved server-side.

Vulnerability ID	Vulnerability-464283
Vulnerability Title	TLS certificate
Vulnerability Description from Scan Results	TLS (or SSL) helps to protect the confidentiality and integrity of information in transit between the browser and server, and to provide authentication of the server's identity. To serve this purpose, the server must present an TLS certificate that is valid for the server's hostname, is

	issued by a trusted authority and is valid for the current date. If any one of these requirements is not met, TLS connections to the server will not provide the full protection for which TLS is designed. It should be noted that various attacks exist against TLS in general, and in the context of HTTPS web connections in particular. It may be possible for a determined and suitably-positioned attacker to compromise TLS connections without user detection even when a valid TLS certificate is used.
Auditor Recommendation from Scan Results	Configure the application to use a valid and trusted TLS certificate that matches the server hostname and is within its validity period. Enforce HTTPS for all communications, disable deprecated SSL/TLS protocols, enable TLS 1.2/1.3, implement strong cipher suites and HSTS, and establish a robust certificate management process to ensure timely renewal and secure handling of cryptographic keys.
Severity	Medium
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Implemented HTTPS and deployed a valid TLS certificate chaining to the Infosys internal CA (`InfosysCAMCH` via `PPC-CA-NP1` and `Polycloud-CA`), valid July 2026–July 2027 with a correct hostname match.

Vulnerability ID	Vulnerability-464285
Vulnerability Title	Open redirection (DOM-based)
Vulnerability Description from Scan Results	DOM-based vulnerabilities arise when a client-side script reads data from a controllable part of the DOM (for example, the URL) and processes this data in an unsafe way. DOM-based open redirection arises when a script writes controllable data into the target of a redirection in an unsafe way. An attacker may be able to use the vulnerability to construct a URL that, if visited by another application user, will cause a redirection to an arbitrary external domain. This behavior can be leveraged to facilitate phishing attacks against users of the application. The ability to use an authentic application URL, targeting the correct domain and with a valid SSL certificate (if SSL is used), lends credibility to the phishing attack because many users, even if they verify these features, will not notice the subsequent redirection to a different domain.
Auditor Recommendation from Scan Results	The most effective way to avoid DOM-based open redirection vulnerabilities is not to dynamically set redirection targets using data that originated from any untrusted source. If the desired functionality of the application means that this behavior is unavoidable, then defenses must be implemented within the client-side code to prevent malicious data from introducing an arbitrary URL as a redirection target. In general, this is best achieved by using a whitelist of URLs that are permitted redirection

	targets, and strictly validating the target against this list before performing the redirection
Severity	Medium
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Fixed the redirect target to a safe same-origin value and added sanitisation that strips unsafe characters from any URL-derived input.

Vulnerability ID	Vulnerability-464295
Vulnerability Title	Content security policy: allows untrusted style execution
Vulnerability Description from Scan Results	Content Security Policy (CSP) is a security mechanism designed to mitigate cross-site scripting attacks by disabling dangerous behaviours such as untrusted JavaScript execution. Websites can specify their security policy in a response header or meta tag, enabling fine-grained control over dangerous features like scripts and stylesheets.
Auditor Recommendation from Scan Results	Mitigate style-based data exfiltration by avoiding 'unsafe-inline', data: URLs, and global wildcards in style directives. Use a secure, random nonce of at least 8 characters 'nonce RANDOM' in the relevant directive.
Severity	Medium
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	
Corrective Actions Taken	

Vulnerability ID	Vulnerability-464297
Vulnerability Title	Concurrent Session Login
Vulnerability Description from Scan Results	The application allows the same user account to maintain multiple active sessions simultaneously across different browsers, devices, or locations without enforcing session restrictions. This may increase the risk of unauthorized access if account credentials are compromised, shared, or misused, as multiple users can access the application concurrently using the same account.
Auditor Recommendation from Scan Results	• Implement controls to restrict concurrent sessions for the same user account based on business requirements. • Invalidate previous sessions when a new login is established, where applicable. • Maintain a centralized session store to track active user sessions. • Provide session management capabilities such as session termination, logout from all devices, and active session monitoring. • Generate alerts or require re-authentication when concurrent logins are detected from different locations, devices, or IP addresses. • Enforce strong authentication controls such as MFA to reduce the risk of

	unauthorized concurrent access. • Periodically review session management controls and logging mechanisms.
Severity	Medium
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Enabled single-session enforcement in `auth/session_registry.py` and `auth/middleware.py`, rejecting a superseded session with `401 SESSION_SUPERSEDED` and a revoked one with `401 SESSION_REVOKED`.

Vulnerability ID	Vulnerability-464298
Vulnerability Title	Content Security Policy Allows Untrusted Script Execution
Vulnerability Description from Scan Results	The Content Security Policy permits execution of inline scripts and dynamically evaluated JavaScript through the use of: • 'unsafe-inline' • 'unsafe-eval' These directives weaken the browser's protection against Cross-Site Scripting (XSS) attacks by allowing execution of inline code and dynamically generated scripts. An attacker who successfully injects malicious JavaScript may be able to execute arbitrary code within the user's browser context.
Auditor Recommendation from Scan Results	• Implement a strict Content-Security-Policy header for all application responses. • Define approved sources for scripts, styles, images, fonts, frames, and network connections. • Restrict script execution to trusted domains only. • Use CSP nonces or hashes instead of allowing inline scripts. • Configure the frame-ancestors directive to mitigate Clickjacking attacks. • Regularly review and update the CSP policy to align with application functionality and security best practices.
Severity	Medium
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	
Corrective Actions Taken	

Vulnerability ID	Vulnerability-464301
Vulnerability Title	Hardcoded / Default Credentials Exposure
Vulnerability Description from Scan Results	The application appears to expose credentials within a client-side request parameter. The presence of values resembling usernames and passwords (for example, ITLINFOSYSGuest and Imap@4012$) may indicate the use of hardcoded, default, or embedded credentials within the application workflow. If these credentials are valid, shared, or used for backend connectivity, an attacker may leverage them to gain unauthorized access to application components or supporting systems.
Auditor Recommendation from Scan Results	

	1. Verify whether the exposed values are active credentials or test data. 2. Remove all hardcoded credentials from application code and client-side requests. 3. Store secrets securely using: o Secure vaults o Environment variables o Managed identity solutions 4. Implement credential rotation. 5. Use role-based service accounts with least privilege access. 6. Never expose authentication secrets to browser clients.
Severity	Medium
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Removed the default credential from the configuration and documentation files, added a guard in `auth/seed_admin.py`, and rotated the live administrator credential.

Vulnerability ID	Vulnerability-464304
Vulnerability Title	Sensitive Information Stored in Session Storage
Vulnerability Description from Scan Results	The application stores a JWT access token containing authenticated user information in the browser's Session Storage. Although Session Storage is cleared when the browser tab or session is closed, the stored token remains accessible to JavaScript executing within the application's origin. If an attacker successfully exploits a Cross-Site Scripting (XSS) vulnerability, the token may be accessed and used to impersonate legitimate users, potentially leading to unauthorized access to application resources. In the observed case, the JWT contains information such as: • User Email • Role (admin) • Tenant Information • Authorization Claims These values can be decoded by anyone possessing the token because JWT payloads are Base64 encoded and not encrypted.
Auditor Recommendation from Scan Results	• Avoid storing authentication tokens in Session Storage where possible. • Store session identifiers in Secure, HttpOnly cookies. • Minimize the information stored within JWT payloads. • Avoid including sensitive user or authorization data unless required. • Implement short-lived access tokens and token rotation mechanisms. • Enforce a strong Content Security Policy (CSP) and XSS protections. • Invalidate tokens upon logout and implement server-side token revocation where applicable.
Severity	Medium
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Migrated both pages to the HttpOnly cookie session pattern with no credential read, and blocked backup source files from being served from the static mount.

Vulnerability ID	Vulnerability-464288
Vulnerability Title	HTML5 storage manipulation (DOM-based)
Vulnerability Description from Scan Results	DOM-based vulnerabilities arise when a client-side script reads data from a controllable part of the DOM (for example, the URL) and processes this data in an unsafe way. HTML5 storage manipulation arises when a script stores controllable data in the HTML5 storage of the web browser (either localStorage or sessionStorage). An attacker may be able to use this behavior to construct a URL that, if visited by another application user, will cause the user's browser to store attacker-controllable data. This behavior does not in itself constitute a security vulnerability. However, if the application later reads the data back from storage and processes it in an unsafe way, then an attacker may be able to leverage the storage mechanism to deliver other DOM-based attacks, such as cross-site scripting and JavaScript injection. Burp Suite automatically identifies this issue using dynamic and static code analysis. Static analysis can lead to false positives that are not actually exploitable. If Burp Scanner has not provided any evidence resulting from dynamic analysis, you should review the relevant code and execution paths to determine whether this vulnerability is indeed present, or whether mitigations are in place that would prevent exploitation.
Auditor Recommendation from Scan Results	The most effective way to avoid DOM-based HTML5 storage manipulation is not to place in HTML5 storage any data that originated from any untrusted source. If the desired functionality of the application means that this behavior is unavoidable, then defenses must be implemented within the client-side code to prevent malicious data from being stored.
Severity	Low
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Added validation before saving to browser storage and encoded the value before showing it on the page.

Vulnerability ID	Vulnerability-464290
Vulnerability Title	Email addresses disclosed
Vulnerability Description from Scan Results	The presence of email addresses within application responses does not necessarily constitute a security vulnerability. Email addresses may appear intentionally within contact information, and many applications (such as web mail) include arbitrary third-party email addresses within their core content. However, email addresses of developers and other individuals (whether appearing on-screen or hidden within page source) may disclose information that is useful to an attacker; for example, they

	may represent usernames that can be used at the application's login, and they may be used in social engineering attacks against the organization's personnel. Unnecessary or excessive disclosure of email addresses may also lead to an increase in the volume of spam email received.
Auditor Recommendation from Scan Results	Consider removing any email addresses that are unnecessary, or replacing personal addresses with anonymous mailbox addresses (such as helpdesk@example.com). To reduce the quantity of spam sent to anonymous mailbox addresses, consider hiding the email address and instead providing a form that generates the email server-side, protected by a CAPTCHA if necessary.
Severity	Low
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	No code change required.

Vulnerability ID	Vulnerability-464291
Vulnerability Title	Cross-origin resource sharing: arbitrary origin trusted
Vulnerability Description from Scan Results	An HTML5 cross-origin resource sharing (CORS) policy controls whether and how content running on other domains can perform two-way interaction with the domain that publishes the policy. The policy is fine-grained and can apply access controls per-request based on the URL and other features of the request. Trusting arbitrary origins effectively disables the same-origin policy, allowing two-way interaction by third-party web sites. Unless the response consists only of unprotected public content, this policy is likely to present a security risk. If the site specifies the header Access-Control-Allow-Credentials: true, third-party sites may be able to carry out privileged actions and retrieve sensitive information. Even if it does not, attackers may be able to bypass any IP-based access controls by proxying through users' browsers.
Auditor Recommendation from Scan Results	Rather than using a wildcard or programmatically verifying supplied origins, use a whitelist of trusted domains.
Severity	Low
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Updated the configuration so `allow_credentials` can never be true alongside an empty or wildcard allowlist, and suppressed CORS headers on public static assets via `StaticExemptCORSMiddleware`.

Vulnerability ID	Vulnerability-464296
Vulnerability Title	Content security policy: allows untrusted script execution
Vulnerability Description from Scan Results	Content Security Policy (CSP) is a security mechanism designed to mitigate cross-site scripting attacks by disabling dangerous behaviours such as untrusted JavaScript execution. Websites can specify their security policy in a response header or meta tag, enabling fine-grained control over dangerous features like scripts and stylesheets.
Auditor Recommendation from Scan Results	Mitigate cross-site scripting by avoiding 'unsafe-inline', 'unsafe-eval', data: URLs, and global wildcards in script directives. Use a secure, random nonce of at least 8 characters 'nonce RANDOM' to prevent untrusted JavaScript execution.
Severity	Low
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	
Corrective Actions Taken	

Vulnerability ID	Vulnerability-464299
Vulnerability Title	Cookie Attribute: Persistent Cookie
Vulnerability Description from Scan Results	The application utilizes a persistent authentication cookie (.AspNet.ApplicationCookie) that remains stored in the browser beyond the active user session. Persistent cookies continue to exist until their configured expiration date, increasing the risk of unauthorized access if the user's device is shared, lost, or compromised. Long-lived authentication cookies may facilitate session reuse by unauthorized users and increase the impact of cookie theft attacks.
Auditor Recommendation from Scan Results	• Use session cookies for authentication wherever feasible. • Configure shorter expiration periods for authentication cookies. • Implement: o Secure flag o HttpOnly flag o SameSite attribute • Enforce server-side session timeout and re-authentication for sensitive operations. • Invalidate authentication cookies upon logout. • Consider implementing idle session timeout and absolute session lifetime controls.
Severity	Low
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Set `HttpOnly`, `Secure` and `SameSite=Lax` on all session cookies in `routers/auth.py::_token_response`.

Vulnerability ID	Vulnerability-464300
Vulnerability Title	COOKIE Attributes Not Set Properly

Vulnerability Description from Scan Results	The application is observed to set cookies without adequate security attributes, which weakens the protection of session and user data. Secure cookie attributes such as Secure, HttpOnly, and SameSite are critical controls that help protect cookies from common client-side attacks. If these attributes are not properly configured, cookies may be exposed to: • Client-side script access (XSS exploitation) • Transmission over insecure channels (HTTP) • Cross-Site Request Forgery (CSRF) attacks
Auditor Recommendation from Scan Results	• Enable Secure Attribute and Ensure cookies are only transmitted over HTTPS:- Set-Cookie: sessionId=xyz; Secure • Enable HttpOnly Attribute and Prevent access to cookies via client-side scripts: Set-Cookie: sessionId=xyz; HttpOnly • Set SameSite Attribute and Protect against CSRF attacks: Set-Cookie: sessionId=xyz; SameSite=Strict
Severity	Low
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Set `HttpOnly`, `Secure` and `SameSite=Lax` on all session cookies in `routers/auth.py::_token_response`.

Vulnerability ID	Vulnerability-464305
Vulnerability Title	Session Time Out
Vulnerability Description from Scan Results	Application session is not timing out after certain period of idle time.
Auditor Recommendation from Scan Results	• Users must be forcefully logged out, and their session should be terminated at the server after a certain period of idle time. • Configure the session time out to 20 minutes or to the minimal time as per the Business requirement.
Severity	Low
Overall Status	Closed
IT Owner	Kanade, Trupti
Status	Closed
Assessor Comment	The issue is acknowledged and fixed by the application team. Hence closed.
Corrective Actions Taken	Added a client-side idle timeout (18-minute warning, 20-minute forced logout) in `static/session-controls.js` and enforced a 12-hour absolute session lifetime via `AUTH_SESSION_MAX_HOURS`.

CONCLUSION

- The vulnerability assessment on **"AgentOS"** application has been carried out. On the whole, Project Team fixed and closed all the vulnerabilities.

- Above mentioned vulnerabilities are the result of one cycle of vulnerability assessment.

REFERENCES

http://www.owasp.org

http://www.securityfocus.com/

http://cwe.mitre.org/

This documentation is the property of Infosys Limited. All ideas and information contained within these documents are the intellectual property rights of Infosys Limited. These documents are not for general distribution and are meant for use only for the person they are specifically issued to. Copying or unauthorized distribution of these documents, in any form or means including electronic, mechanical, photocopying or otherwise is illegal.